DESPLIEGUE DE APLICACIONES WEB

# UT 2. Configuración y administración de servidores web

*Bloque 1 · Configuración avanzada del servidor web*

# Qué vamos a ver

**1**

**Configuración avanzada del servidor web**

*Arquitectura y directivas*

**2**

**Módulos: instalación, configuración y uso**

*Apache modular*

**3**

**Hosts virtuales**

*Creación, configuración y utilización*

**4**

**Autenticación y control de acceso**

*Require, .htaccess, grupos*

**5**

**El protocolo HTTPS**

*TLS 1.2 / TLS 1.3*

**6**

**Certificados y servidores de certificados**

*Autofirmados y Let's Encrypt*

**7**

**Despliegue de aplicaciones sobre servidores web**

*PHP, permisos, pruebas*

**8**

**Navegadores web: apariencia y uso**

*DevTools, idioma, certificados*

**9**

**Registro y monitorización**

*Logs, mod_status, herramientas*

RESULTADO DE APRENDIZAJE

# RA2 y criterios de evaluación

| Criterio | Qué se evalúa |
|---|---|
| a | Se han reconocido los parámetros de administración más importantes del servidor web. |
| b | Se ha ampliado la funcionalidad del servidor mediante la activación y configuración de módulos. |
| c | Se han creado y configurado sitios virtuales. |
| d | Se han configurado los mecanismos de autenticación y control de acceso del servidor. |
| e | Se han obtenido e instalado certificados digitales. |
| f | Se han establecido mecanismos para asegurar las comunicaciones entre cliente y servidor. |
| g | Se han realizado pruebas de funcionamiento y rendimiento del servidor web. |
| h | Se ha elaborado documentación relativa a la configuración, administración segura y recomendaciones de uso. |
| i | Se han realizado los ajustes necesarios para la implantación de aplicaciones en el servidor web. |

BLOQUE 1

# Configuración avanzada del servidor web

*Arquitectura de Apache 2.4 y las directivas principales del archivo de configuración, explicadas con el detalle de un manual.*

- Apache 2.4 hoy y un poco de historia
- El archivo de configuración y su organización
- DocumentRoot, Directory y la herencia de permisos
- Alias y negociación de contenido
- Arquitectura: los módulos MPM
- Directivas del servidor, paso a paso
- Seguridad básica y filtrado con FilesMatch

# ¿Qué es Apache y por qué lo seguimos usando?

Un servidor web es, en esencia, un programa que permanece a la escucha de peticiones HTTP y responde a ellas entregando páginas, imágenes o cualquier otro recurso. Apache HTTP Server —normalmente llamado solo «Apache»— es, junto con Nginx, uno de los dos servidores web más usados del planeta, y lleva ocupando ese puesto desde mediados de los años noventa.

Su éxito se explica por tres razones que siguen vigentes hoy: es software libre (se puede instalar, estudiar y modificar sin coste de licencia), es modular (incorpora solo las piezas que necesitamos, activando o desactivando módulos) y es multiplataforma (funciona igual de bien en Linux, Windows o macOS). Esa combinación lo convirtió en la base de la mayoría de los alojamientos web durante más de veinte años.

A lo largo de esta unidad trabajaremos siempre sobre la rama 2.4, que es la única que recibe mantenimiento activo en la actualidad; la versión estable en la fecha de estos apuntes es la 2.4.69 (octubre de 2026). Las ramas 2.2 y anteriores están fuera de soporte —ya no reciben parches de seguridad— por lo que jamás deberían usarse en un servidor de producción.

Todo lo que veremos se apoya en la instalación por paquete de Apache 2.4 sobre Ubuntu 24.04 LTS (igualmente válido sobre Ubuntu 26.04 LTS). Podemos comprobar en cualquier momento qué versión tenemos instalada con un comando muy sencillo.

*sudo apache2 -v  /  sudo apache2ctl -V  (esta segunda opción muestra además el MPM compilado y las rutas de compilación)*

# De la versión 1.3 a la 2.4: por qué cambió la arquitectura

Para entender por qué Apache funciona como funciona hoy conviene mirar atrás. Hasta la versión 1.3, el servidor se apoyaba en un modelo muy simple: un proceso «padre» se limitaba a vigilar y crear procesos «hijo» según hacía falta, y cada proceso hijo atendía una conexión completa él solo. Era un diseño fácil de entender, pero poco flexible.

Ese modelo funcionaba razonablemente bien en sistemas tipo Unix, pero daba problemas serios en otros sistemas operativos —Windows, sobre todo— donde crear y destruir procesos es una operación mucho más costosa que crear y destruir hilos. Apache necesitaba una forma de adaptar su estrategia de concurrencia a cada sistema operativo sin reescribir el núcleo del servidor para cada uno.

La solución llegó con la versión 2.0: se introdujeron los módulos multiproceso, los MPM (MultiProcessing Modules), que aislaron «cómo se atienden las conexiones» del resto del servidor. A partir de ese momento, además, se incorporaron otros módulos importantes que ampliaron mucho las posibilidades de configuración. La siguiente versión relevante fue la 2.2, que todavía hoy se encuentra en bastantes sistemas antiguos; estos apuntes se basan íntegramente en la 2.4, que perfecciona ese mismo diseño modular.

# MPM: quién decide cómo se atienden las peticiones

Un MPM (MultiProcessing Module) es el módulo responsable de decidir cómo genera Apache los procesos o hilos que atienden las peticiones entrantes. Es una decisión especialmente delicada porque de ella depende directamente el rendimiento del servidor bajo carga: no es lo mismo atender diez peticiones por segundo que diez mil.

Solo puede haber un MPM activo a la vez, porque el servidor se compila (o se configura, según la instalación) para trabajar con uno en concreto. Apache 2.4 ofrece varios: prefork, worker, event e itk son los más habituales en una distribución como Ubuntu.

A los procesos que Apache crea para atender peticiones se les llama, siguiendo la terminología de Unix, fork (procesos hijo); a las unidades de ejecución más ligeras que pueden convivir dentro de un mismo proceso se las llama hilos o threads. La diferencia no es solo de vocabulario: los hilos comparten la memoria del proceso que los lanza, mientras que los procesos hijo son completamente independientes entre sí. Esa independencia hace que los procesos sean más robustos ante fallos —si uno se cae, no arrastra a los demás— pero también más lentos de crear y más pesados en memoria que los hilos.

# Prefork frente a worker / event

Los tres MPM más usados resuelven el mismo problema —atender muchas conexiones a la vez— de dos maneras muy distintas:




Cada petición la atiende un proceso completo e independiente. Muy estable y compatible con módulos antiguos no preparados para hilos (p. ej. mod_php clásico), pero cada proceso consume más memoria.

Pocos procesos, cada uno con varios hilos que comparten memoria y atienden muchas conexiones a la vez. Mucho más eficiente en tráfico alto; event (el MPM activado por defecto desde Apache 2.4) además separa la gestión de conexiones inactivas en keep-alive.

*Las peticiones entrantes las atienden procesos o hilos según cuál sea el MPM compilado o activado: la elección condiciona el rendimiento, nunca el resultado que ve el visitante.*

# Consultar y cambiar el MPM activo

Podemos averiguar en cualquier momento qué MPM está usando nuestro Apache con la opción -V de apache2ctl, que entre otros datos muestra el módulo compilado. Al instalar Apache 2.4 en Ubuntu, el MPM event queda activo por defecto porque es el más eficiente para el tráfico habitual de un servidor moderno.

```
# Ver el MPM activo
sudo apache2ctl -V | grep MPM

# Cambiar de MPM: en Ubuntu moderno NO son paquetes separados,
# son módulos que se activan y desactivan como cualquier otro
sudo a2dismod mpm_event
sudo a2enmod mpm_prefork
sudo systemctl restart apache2
```

*Importante (y a menudo origen de confusión): en versiones antiguas de Ubuntu existían paquetes independientes apache2-mpm-prefork / -worker / -event que había que instalar o desinstalar. Desde Ubuntu 16.04, el MPM se gestiona exactamente igual que el resto de módulos, con a2enmod / a2dismod.*

# apache2.conf: el punto de partida

Toda la personalidad de nuestro servidor —qué carpeta sirve, qué módulos carga, cómo responde a cada petición— se define en un archivo de texto plano. En la instalación por paquete de Ubuntu ese archivo es /etc/apache2/apache2.conf; en otras distribuciones, o si compilamos Apache desde cero, el archivo equivalente puede llamarse httpd.conf (el nombre del archivo principal se decide al compilar el servidor, y Ubuntu simplemente lo ha renombrado).

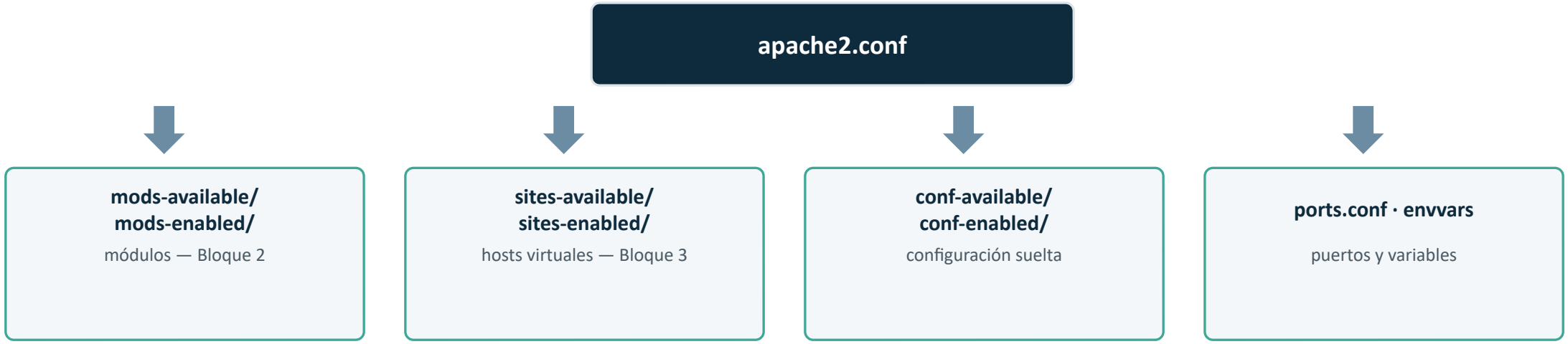
El archivo mezcla dos tipos de líneas: las directivas, que son instrucciones reales que el servidor interpreta y ejecuta, y los comentarios, que son líneas que empiezan por el símbolo #. Apache ignora por completo los comentarios y las líneas en blanco; están ahí únicamente para que la persona que administra el servidor entienda mejor cada sección.

Antes de tocar cualquier archivo de configuración —no solo este— conviene adquirir el hábito de hacer una copia de seguridad. Es una norma que nos va a ahorrar más de un disgusto a lo largo del curso: basta un simple cp para poder volver atrás si algo sale mal.

*sudo cp apache2.conf apache2.conf.bak*

# Un único archivo no basta: Include

Al final de apache2.conf encontramos varias directivas Include que reparten la configuración en piezas más pequeñas y manejables:




*Dividir la configuración facilita mantenerla: cada pieza (un módulo, un sitio, una prueba) se activa o desactiva de forma independiente, sin arriesgar el resto del archivo ni perder la posibilidad de revertir un cambio con facilidad.*

# Directivas generales del proceso

ServerRoot indica el directorio raíz desde el que se resuelven las rutas relativas que aparecen en el resto de la configuración. Es un error muy habitual confundirlo con el directorio donde colocamos las páginas web: ServerRoot no tiene nada que ver con el contenido que servimos, solo con dónde vive la propia instalación de Apache.

PidFile establece la ruta del archivo donde el proceso principal de Apache guarda su identificador de proceso (PID). Por defecto se escribe en /var/run/apache2, y ese valor en realidad se construye a partir de una constante —APACHE_PID_FILE— definida en /etc/apache2/envvars; es un buen ejemplo de cómo el archivo principal se apoya en variables definidas en otro sitio para no repetir rutas por todas partes.

Timeout indica cuántos segundos espera el servidor una respuesta antes de dar por perdida una conexión y cancelarla. El valor por defecto son 300 segundos, y la recomendación general es no tocarlo salvo que tengamos una razón de peso: bajarlo demasiado puede cortar conexiones lentas pero legítimas, y subirlo deja recursos bloqueados más tiempo del necesario.

# KeepAlive: por qué importa reutilizar la conexión

Una página típica no es un solo archivo: es el HTML más sus imágenes, su CSS y su JavaScript. Sin KeepAlive, el navegador tendría que abrir y cerrar una conexión TCP distinta para cada uno de esos recursos.




*4 peticiones → 4 conexiones TCP abiertas y cerradas, una a una.*

*Misma conexión reutilizada para las 4 peticiones: menos trabajo de apertura/cierre, respuesta más rápida.*

**KeepAlive** On u Off: activa o desactiva esta reutilización de conexión.

**KeepAliveTimeout** segundos que el servidor mantiene la conexión abierta esperando otra petición del mismo cliente antes de cerrarla.

**MaxKeepAliveRequests** número máximo de peticiones que se atienden por conexión antes de obligar a reabrirla.

*Un KeepAliveTimeout alto (15 s, por ejemplo) es contraproducente: rara vez una página tarda tanto en terminar de pedir todos sus recursos, y mientras tanto el servidor mantiene conexiones ociosas sin reutilizar. Valores de 2 a 5 segundos suelen ser razonables.*

# Resumen rápido

| Directiva | Para qué sirve | Valor habitual |
|---|---|---|
| ServerRoot | Raíz desde la que se resuelven las rutas relativas de la configuración (no es la carpeta de las páginas web). | /etc/apache2 |
| PidFile | Ruta donde se guarda el PID del proceso principal. | /var/run/apache2 |
| Timeout | Segundos antes de cancelar una conexión sin respuesta. | 300 s |
| KeepAlive* | Reutilización de la conexión entre peticiones del mismo cliente. | On, 2–5 s |

# Identidad y escucha del servidor

Listen obliga a Apache a escuchar en direcciones IP y puertos concretos; vive en /etc/apache2/ports.conf, separado del archivo principal porque es una pieza que conviene poder revisar de un vistazo. Si solo se indica un número de puerto, el servidor escucha en ese puerto en todas las interfaces de red de la máquina; si se indica también una IP, se restringe a esa interfaz concreta. Esta misma directiva puede combinarse más adelante con hosts virtuales para que Apache responda de forma distinta según la IP, el nombre o el puerto por el que llegue la petición.

User y Group determinan con qué usuario y grupo del sistema se ejecutan los procesos hijo que atienden las peticiones; en Ubuntu el valor por defecto es www-data para ambos. Es una decisión de seguridad, no un detalle menor: esos procesos jamás deben lanzarse como root, porque cualquier vulnerabilidad en una aplicación web se convertiría automáticamente en una vía de acceso total al sistema. Como consecuencia práctica, cualquier archivo al que www-data no tenga permiso de lectura será también invisible para quien visite la web.

ServerAdmin fija el correo de contacto que Apache muestra en las páginas de error generadas automáticamente; en un servidor real debería ser una dirección que alguien revise de verdad. ServerName, por su parte, establece el nombre del servidor y debe cumplir el formato de un nombre DNS. Mientras no dispongamos de un dominio propio, podemos usar el nombre de la propia máquina virtual —el que aparece en /etc/hostname— como ServerName provisional.

*Listen, ServerAdmin y ServerName suelen repetirse, con valores propios, dentro de cada host virtual: lo veremos con detalle en el Bloque 3.*

# Resumen rápido

| Directiva | Para qué sirve | Dónde vive |
|---|---|---|
| Listen | Fuerza a Apache a escuchar en IPs/puertos concretos. | ports.conf |
| User / Group | Usuario y grupo de los procesos hijo (nunca root). | apache2.conf |
| ServerAdmin | Correo de contacto mostrado en páginas de error. | sites-available/*.conf |
| ServerName | Nombre del servidor (formato DNS). | sites-available/*.conf |

# DocumentRoot y Directory

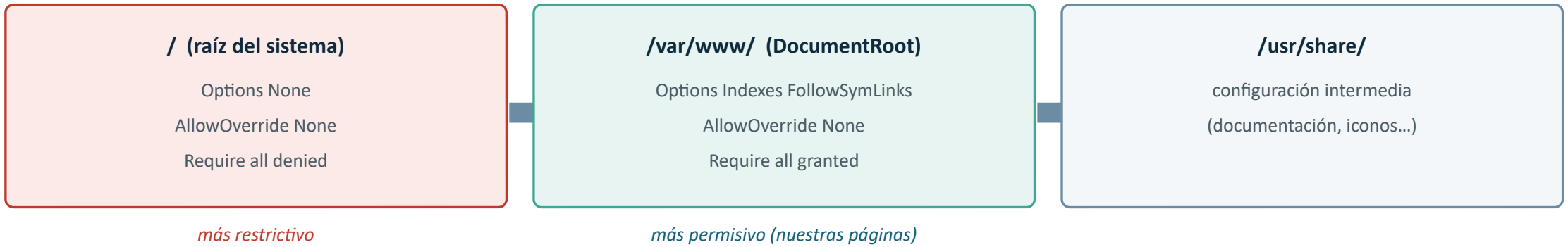
DocumentRoot indica cuál es la carpeta raíz donde colocamos las páginas del sitio; por defecto, en Ubuntu, es /var/www/html. Todo lo que el visitante pide con una ruta relativa —por ejemplo /galeria/foto1.jpg— se busca dentro de esa carpeta, de modo que /galeria/foto1.jpg se traduce internamente en /var/www/html/galeria/foto1.jpg.

Directory es la directiva que configura qué comportamiento se permite para un directorio concreto del sistema de archivos: si se puede listar su contenido, si se siguen los enlaces simbólicos, si se permite sobrescribir la configuración con un .htaccess, quién tiene permiso para entrar. Lo importante es que esa configuración se hereda en todos los subdirectorios, a menos que una definición más concreta la sobrescriba explícitamente.

Dentro de apache2.conf la etiqueta Directory aparece como mínimo tres veces, y ese orden no es casual: cuanto más general es el directorio, más restrictiva es la configuración, y cuanto más concreto, más permisos se conceden. Lo vemos en detalle en el diagrama de la página siguiente.

# De lo general a lo particular

apache2.conf define tres bloques <Directory>, cada vez más permisivos a medida que apuntan a un directorio más concreto:




```
<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
```

*Options controla funciones como listar el directorio (Indexes) o seguir enlaces simbólicos; AllowOverride decide si se permite usar .htaccess (Bloque 4); Require controla quién puede entrar.*

# DirectoryIndex y AccessFileName

Cuando alguien visita una URL que apunta a un directorio en lugar de a un archivo concreto —por ejemplo, http://localhost/ sin más— Apache necesita saber qué archivo servir por defecto. Eso es justo lo que decide DirectoryIndex, que por defecto vale index.html.

Esta directiva admite en realidad una lista de prioridad, no un único nombre: el servidor prueba cada nombre en el orden en que aparecen hasta encontrar uno que exista. Por ejemplo, DirectoryIndex comienzo.html index.html haría que Apache sirviera comienzo.html si existe, y solo recurriría a index.html en caso contrario.

Si ninguno de los nombres de la lista existe en el directorio solicitado, y además la opción Indexes está activa en ese Directory, Apache genera automáticamente un listado del contenido de la carpeta. Conviene tenerlo presente porque es, en la práctica, un riesgo de seguridad si no se desea exponer esa información: revela la estructura interna del sitio a cualquier visitante.

AccessFileName, por último, fija el nombre que debe tener el archivo de configuración distribuida por directorio —por defecto, .htaccess—. Volveremos sobre él con detalle en el Bloque 4, cuando veamos autenticación y control de acceso.

EJERCICIO 1

# DirectoryIndex en acción

- Crea una página comienzo.html en /var/www/html/, junto a un index.html ya existente.
- Modifica la directiva DirectoryIndex para que comienzo.html se sirva antes que index.html al visitar http://localhost.
- Comprueba el resultado en el navegador y deja la configuración como estaba.

# ServerSignature y ServerTokens

Cuando Apache genera una página automáticamente —un error 404, un listado de directorio— puede añadir al final una pequeña firma con información sobre sí mismo. Eso es exactamente lo que controla ServerSignature: con el valor On añade esa firma; con Off, no.

ServerTokens, por su parte, regula cuánta información entrega Apache en la cabecera HTTP Server de cada respuesta, desde Full —que revela versión del servidor, sistema operativo y hasta los módulos instalados— hasta Prod, que se limita a decir «Apache» y nada más. Ambas directivas se encuentran en /etc/apache2/conf-available/security.conf.

La razón por la que esto importa no es estética: cuanta más información técnica expone un servidor sobre sí mismo, más fácil se lo ponemos a un atacante para identificar versiones concretas con vulnerabilidades conocidas y dirigir su ataque directamente a ellas. Por eso la buena práctica de seguridad es reducir ambas directivas al mínimo —ServerTokens Prod y ServerSignature Off— incluso en un servidor de pruebas: adquirir esa costumbre desde ya evita tener que acordarse de hacerlo el día en que el servidor pase a producción.

# FilesMatch y expresiones regulares (resumen)

*FilesMatch aplica directivas a los archivos cuyo nombre coincida con un patrón de expresión regular. Antes de usarla conviene repasar los símbolos más habituales:*

| Símbolo | Significado | Ejemplo |
|---|---|---|
| . ^ $ | cualquier carácter · inicio · fin de línea | ^foto.*\.jpg$ |
| * + ? | 0 o más · 1 o más · 0 o 1 repeticiones | ka+ coincide con ka, kaa… |
| [ ] [^ ] | conjunto de caracteres · negación del conjunto | [0-9]{3} → tres dígitos |
| ( ) \| | agrupación · disyunción ("o") | ^(pdf\|doc)$ |
| {n} {n,} {n,m} | número exacto · mínimo · rango de repeticiones | [0-9]{3,4} |

*FilesMatch se define en /etc/apache2/apache2.conf (o en el host virtual que corresponda) y acepta cualquiera de estos patrones.*

# FilesMatch aplicado: bloquear imágenes .gif y .png

Con este bloque en la configuración, cada petición de archivo se compara contra el patrón antes de decidir si se sirve:

```
<FilesMatch "\.(gif|png)$">   Require all denied   </FilesMatch>
```

**foto.jpg**

no coincide

✓ se sirve con normalidad

**logo.png**

coincide con el patrón

✕ Require all denied

**banner.gif**

coincide con el patrón

✕ Require all denied

**estilo.css**

no coincide

✓ se sirve con normalidad

*El patrón solo mira el nombre del archivo, no su contenido real: un .gif renombrado a .jpg se serviría igualmente, y viceversa.*

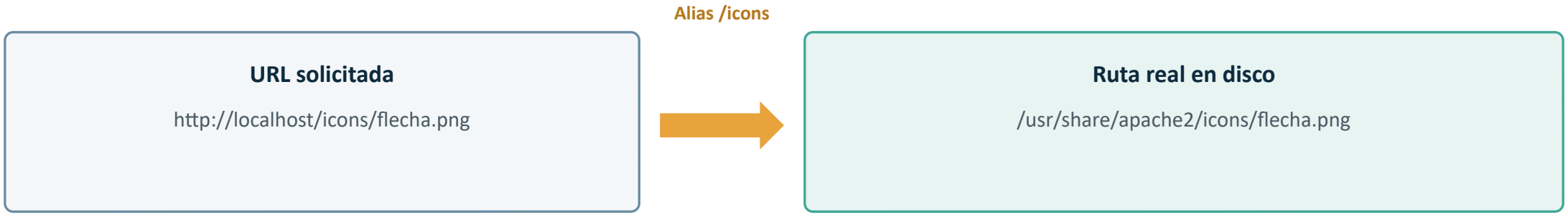
EJERCICIO 2

# Bloquear imágenes con FilesMatch

- Crea una página con dos imágenes: una .gif y otra .jpg, y publícala en el servidor.
- Usa la directiva FilesMatch para denegar el acceso a todas las imágenes .gif.
- Comprueba que solo se ve la imagen .jpg (vacía la caché del navegador si hace falta) y deshaz el cambio al terminar.

# Alias y ScriptAlias: servir contenido fuera de DocumentRoot

Alias asocia una URL a un directorio que puede estar en cualquier parte del sistema de archivos, sin moverlo ni enlazarlo físicamente. ScriptAlias hace exactamente lo mismo pero además marca ese directorio como ejecutable (scripts CGI).




*La carpeta /usr/share/apache2/icons/ está completamente fuera de /var/www/html, pero el navegador nunca lo nota: para él, /icons/ es una ruta más del sitio.*

**LanguagePriority** y **negotiation_module** permiten que el servidor elija el idioma de una página (index.html.es / index.html.en…) según la configuración del navegador del cliente.

# Elegir idioma y codificación automáticamente

La negociación de contenido es el mecanismo por el cual Apache decide, sin que el visitante tenga que hacer nada, qué versión de una página entregarle entre varias disponibles. El caso más habitual es el idioma: si existen saludo.html.es, saludo.html.en y saludo.html.de, el servidor puede escoger automáticamente la que mejor encaje con la configuración de idioma del navegador que hace la petición.

LanguagePriority establece qué idioma usar como respaldo cuando el navegador del cliente no indica ninguna preferencia reconocible, o cuando varias opciones empatan. Es importante fijar siempre un idioma por defecto razonable —español, en nuestro caso— para que la página nunca se quede sin mostrar nada.

AddDefaultCharset fija la codificación de caracteres con la que el servidor etiqueta las respuestas; UTF-8 es el valor por defecto y, hoy en día, el estándar recomendado para prácticamente cualquier sitio web, porque es el único que representa sin ambigüedad cualquier idioma y símbolo.

EJERCICIO 3

# Alias y páginas multi-idioma

- Crea /usr/share/apache2/images/ con un par de imágenes y un Alias /imagenes que apunte ahí; comprueba que una página puede usarlas desde ese alias.
- Crea saludo.html.es, saludo.html.en y saludo.html.de con un saludo distinto en cada idioma.
- Configura LanguagePriority para que, sin idioma indicado por el navegador, se sirva por defecto el español; comprueba el resultado en localhost/saludo.html.